

Data Structures Review

Algorithms and Big O Notation

- **Algorithms:** A set of unambiguous instructions for solving a problem or carrying out a task. Algorithms must finish in a finite number of steps and each step must be precise and unambiguous.
- **Big O Notation:** Describes the worst-case performance, or growth rate, of an algorithm as the input size increases. It focuses on how resource usage grows with input size, ignoring constant factors and lower-order terms.

Common Time Complexities

- **O(1) - Constant Time:** Algorithm takes the same amount of time regardless of input size.

```
1 function checkEvenOrOdd(number) {  
2   if (number % 2 === 0) {  
3     return "Even";  
4   } else {  
5     return "Odd";  
6   }  
7 }  
8  
9 console.log(checkEvenOrOdd(4)) // "Even"  
10 console.log(checkEvenOrOdd(5)) // "Odd"
```

"Even"

"Odd"

- **O(log n) - Logarithmic Time:** Time increases slowly as input grows. Common in algorithms that repeatedly reduce problem size by a fraction (like Binary Search).
- **O(n) - Linear Time:** Running time increases proportionally to input size.

```
for (const grade of grades) { // grades is an array  
  console.log(grade);  
}
```

- **O(n log n) - Log-Linear Time:** Common time complexity of efficient sorting algorithms like Merge Sort and Quick Sort.
- **O(n²) - Quadratic Time:** Running time increases quadratically. Often seen in nested loops.

```
1 const n = 3;  
2  
3 for (let i = 0; i < n; i++) {  
4   for (let j = 0; j < n; j++) {  
5
```



Space Complexity

- **O(1) - Constant Space:** Algorithm uses same amount of memory regardless of input size.
- **O(n) - Linear Space:** Memory usage grows proportionally with input size.
- **O(n²) - Quadratic Space:** Memory usage grows quadratically with input size.

Problem-Solving Techniques

- **Understanding the Problem:** Read the problem statement multiple times. Identify the input, expected output, and how to transform input to output.
- **Pseudocode:** High-level description of algorithm logic that is language-independent. Uses common written language mixed with programming constructs like `IF`, `ELSE`, `FOR`, `WHILE`.

```
GET original_string
SET reversed_string = ""
FOR EACH character IN original_string:
  ADD character TO THE BEGINNING OF reversed_string
DISPLAY reversed_string
```

- **Edge Cases:** Specific, valid inputs that occur at the boundaries of what an algorithm should handle. Always consider and test edge cases.

Arrays

- **Static Arrays:** Have a fixed size determined at initialization. Elements stored in adjacent memory locations. Size cannot be changed during program execution.
- **Dynamic Arrays:** Can grow or shrink automatically during program execution. Handle resizing through automatic copying to larger arrays when needed.

Dynamic Arrays Implementation in JavaScript

1 let numbers = [3, 4, 5, 6];	3
2 // Access elements	
3 console.log(numbers[0]) // 3	[3, 4, 16, 6]
4	
5 // Update elements	
6 numbers[2] = 16	[3, 4, 16, 6, 7]
7 console.log([...numbers]); // [3, 4, 16, 6]	
8	[3, 4, 16, 15, 6, 7]
9 // Add elements	
10 numbers.push(7)	[3, 4, 15, 6, 7]
11 console.log([...numbers]); // [3, 4, 16, 6, 7]	
12	
13 // Add at a specific index	[3, 4, 15, 6]
14 numbers.splice(3, 0, 15);	
15 console.log([...numbers]); // [3, 4, 16, 15, 6, 7]	
16	

Time Complexities for Dynamic Arrays

- **Access:** $O(1)$
- **Insert at end:** $O(1)$ average, $O(n)$ when resizing needed
- **Insert in middle:** $O(n)$
- **Delete:** $O(n)$ for middle, $O(1)$ for end

Stacks

- **Stacks:** Last-In, First-Out (LIFO) data structure. Elements added and removed from the top only.
- **Push Operation:** Adding an element to the top of the stack. Time complexity: $O(1)$.
- **Pop Operation:** Removing an element from the top of the stack. Time complexity: $O(1)$.

```
1 // Using JavaScript array as a stack
2
3 let stack = [];
4
5 // Push operations
6 stack.push(1);
7 stack.push(2);
8 stack.push(3);
9 console.log([...stack]); // [1, 2, 3]
10
11 // Pop operations
12 let topElement = stack.pop();
13 console.log(topElement); // 3
14 console.log([...stack]); // [1, 2]
```

[1, 2, 3]

3

[1, 2]

Queues

- **Queues:** First-In, First-Out (FIFO) data structure. Elements added to the back and removed from the front.
- **Enqueue Operation:** Adding an element to the back of the queue. Time complexity: $O(1)$.
- **Dequeue Operation:** Removing an element from the front of the queue. Time complexity: $O(1)$.

```
1 // Using JavaScript array as a queue
2 let queue = [];
3
4 // Enqueue operations
5 queue.push(1);
6 queue.push(2);
7 queue.push(3);
8 console.log([...queue]); // [1, 2, 3]
9
10 // Dequeue operations
11 let firstElement = queue.shift();
12 console.log(firstElement); // 1
13 console.log([...queue]); // [2, 3]
```

[1, 2, 3]

1

[2, 3]



Linked Lists

- **Linked Lists:** Linear data structure where each node contains data and a reference to the next node. Nodes are connected like a chain.

Singly Linked Lists

- **Structure:** Each node has data and one reference to the next node.
- **Traversal:** Can only move forward from head to tail.
- **Head Node:** First node in the list, usually the only directly accessible node.
- **Tail Node:** Last node in the list, points to `null`.

Operations and Time Complexities

- **Insert at beginning:** $O(1)$
- **Insert at end:** $O(n)$ - must traverse to end
- **Insert in middle:** $O(n)$ - must traverse to position
- **Delete from beginning:** $O(1)$
- **Delete from end:** $O(n)$ - must traverse to find previous node
- **Delete from middle:** $O(n)$ - must traverse to find node

Doubly Linked Lists

- **Structure:** Each node has data and two references: next node and previous node.
- **Traversal:** Can move in both directions.
- **Memory:** Requires more memory than singly linked lists due to extra reference.

When to Use Each Data Structure

- **Arrays (dynamic arrays):** When you need ordered, indexed access and don't know size in advance
- **Stacks:** For LIFO operations (undo functionality, expression evaluation, backtracking)
- **Queues:** For FIFO operations (task scheduling, breadth-first search)
- **Linked Lists:** When frequent insertion/deletion at beginning, unknown size, no random access needed

Assignment

- ☐ Review the Data Structures topics and concepts.

Submit

Ask for Help